



UNIVERSITÀ
DEGLI STUDI
DI PALERMO



Introduzione alle architetture Web

CORSO DI PROGRAMMAZIONE WEB E MOBILE

A.A. 2025/2026

PROF. LUCA CRUCIATA

Sommario

- Elaborazione client-side
- Elaborazione server-side
- Architettura LAMP
- Architettura MEAN e MERN

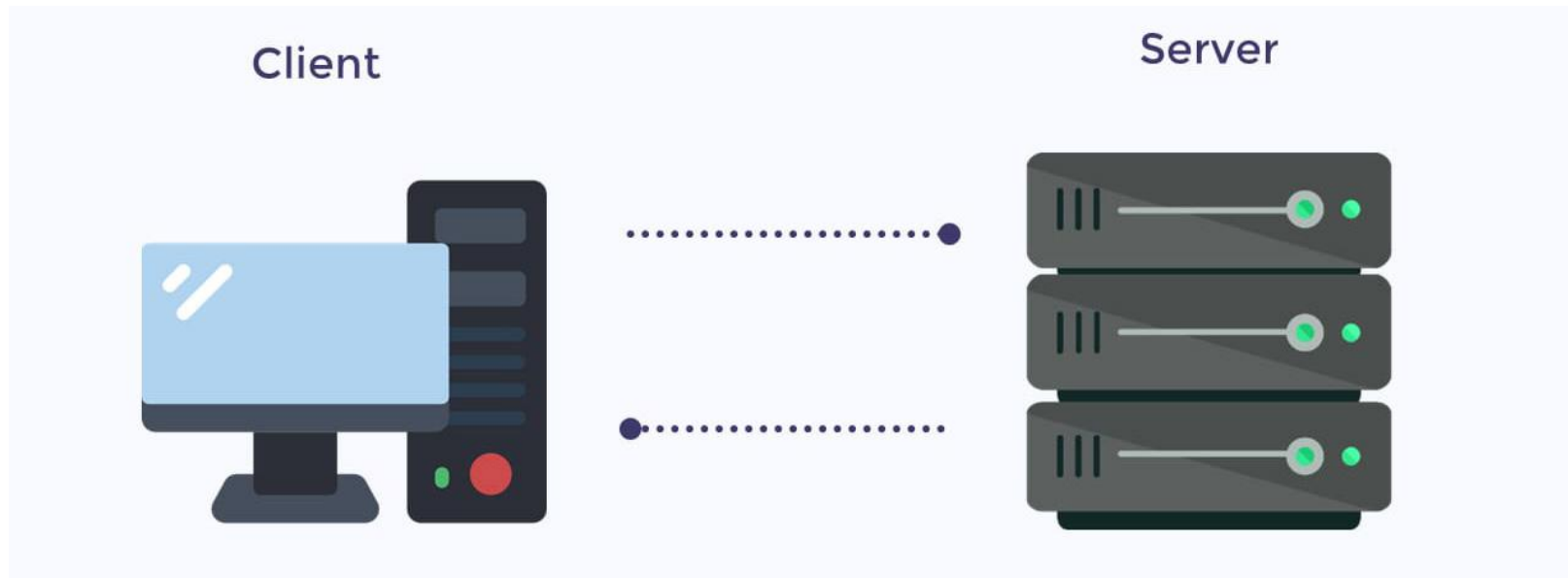
Il web come interfaccia per applicazioni

- La tecnologia WWW viene spesso utilizzata per accedere a delle informazioni e non come ipertesto con fini comunicativi
 - *Sistemi informativi (ad es. consultazione di archivi documentali)*
 - *Carrello elettronico*
 - *Motori di ricerca*
- Solitamente delle pagine con contenuto fisso, indipendente dalle azioni dell'utente (es. pagine Wikipedia) definite **pagine statiche**
- In questo caso si utilizzano quelle che vengono definite **pagine web dinamiche**, perché esibiscono un contenuto che dipende dalle azioni svolte dall'utente attraverso l'interfaccia web.

Client e Server

L'elaborazione di tali pagine web può avvenire:

- *Client side*
- *Server side*



Elaborazione client-side

- Le pagine dinamiche client-side sono pagine HTML che contengono codice sorgente o riferimenti a codice sorgente, eseguito dal browser Internet al momento della ricezione.
- I principali linguaggi utilizzati per questo tipo di applicazioni sono *JavaScript* e *TypeScript*.
 - *Linguaggi di scripting, interpretati*

Client-Side

1. Il browser carica il codice JavaScript dal server.
2. Il codice viene eseguito direttamente sul dispositivo dell'utente.
3. Le interazioni avvengono senza ricaricare la pagina.
4. Se servono dati, il browser li chiede al server tramite API (AJAX, Fetch, WebSocket, ecc.).

L'elaborazione avviene nel browser dell'utente, che si occupa di scaricare il codice ed eseguirlo.

Un sito web in JavaScript con React, Angular o Vue.js.

Elaborazione server-side

- Le pagine dinamiche di tipo server-side consistono nell'esecuzione di programmi sul server web, i quali sono invocati tramite apposite URL e generano come risultato l'elaborazione dati richiesta e la generazione delle pagine HTML dinamiche come risposta al client.
- L'interfaccia **CGI** (*Common Gateway Interface*) definisce la più antica tecnologia per il passaggio dei dati ai programmi sul server: essa consente l'utilizzo di qualunque linguaggio di programmazione.

Server-Side Funzionamento

1. L'utente fa una richiesta (es. clicca su un link).
2. Il server elabora la richiesta (es. legge un database).
3. Il server invia una nuova pagina HTML al browser.
4. Il browser mostra la pagina all'utente.

Vantaggi:

Perfetto per operazioni pesanti (database, sicurezza). Mantiene il codice più segreto e sicuro. Non dipende dal dispositivo dell'utente.

Svantaggi:

Più lento: ogni interazione richiede una richiesta al server. Non interattivo senza ricaricare la pagina.

L'elaborazione avviene interamente sul server remoto, che gestisce anche la logica dell'applicazione e invia al client solo il risultato.

Chi compone la pagina HTML che l'utente vede?

Nelle pagine dinamiche menzionate nella tua slide, l'elaborazione (la logica) e il rendering (la costruzione visiva) sono strettamente collegati.

Il rendering è il processo di trasformazione del codice in un'interfaccia grafica leggibile. A seconda di dove avviene l'elaborazione (**Client-side** o **Server-side**), cambia il momento in cui la pagina viene "costruita".

Il Client-Side Rendering (CSR)

Il rendering della pagina avviene direttamente nel browser dell'utente, anziché sul server.

1. Quando visiti un sito web, il server invia al tuo browser principalmente un file HTML vuoto (o quasi) insieme al file JavaScript.
2. È il **JavaScript** che poi si occupa di "costruire" dinamicamente la pagina web nel tuo browser, caricando e mostrando i dati richiesti.
3. Questo approccio è spesso associato a framework JavaScript moderni, come React, Angular o Vue.

Vantaggi:

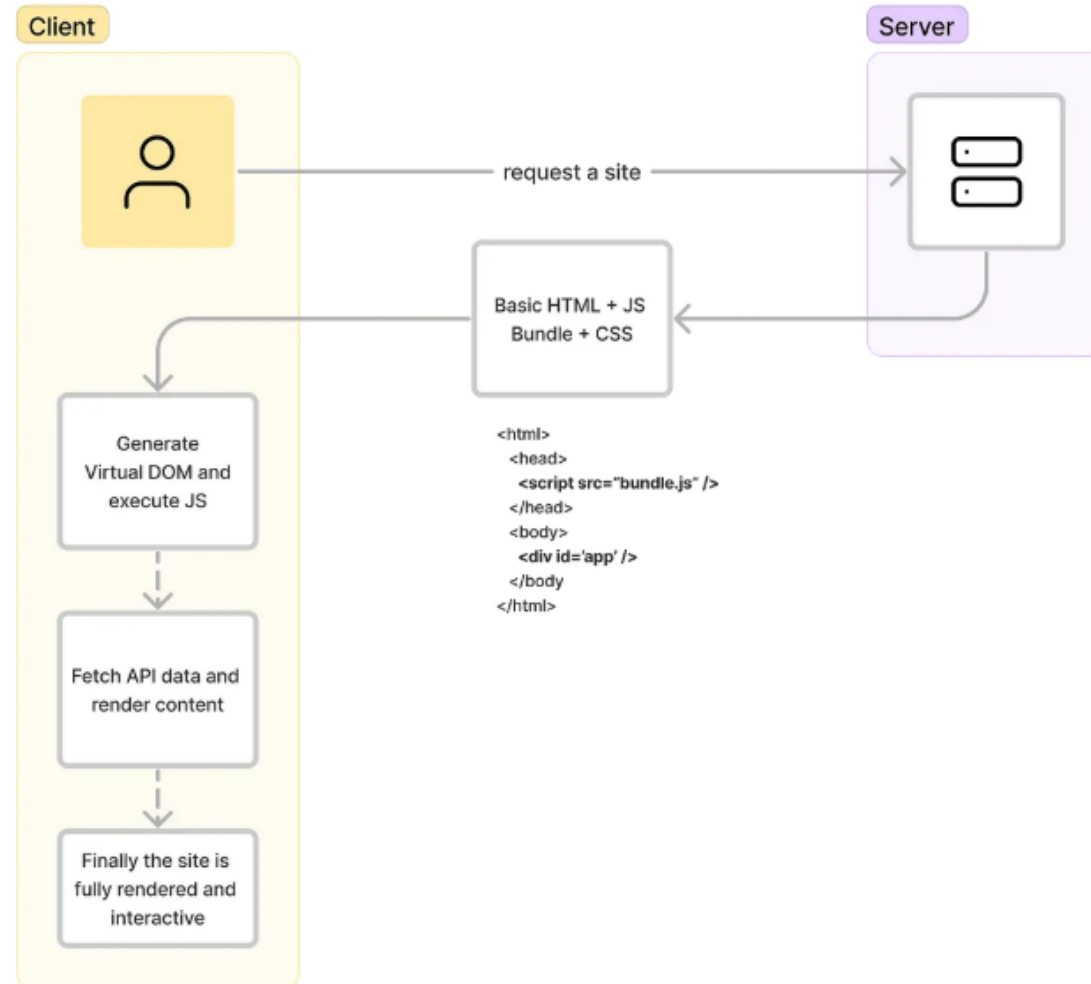
Una volta caricata inizialmente l'applicazione, le interazioni successive con il sito sono più rapide, poiché non è necessario un continuo dialogo con il server.

Svantaggi:

Caricamento più lento, poiché il browser deve scaricare e interpretare molti file JavaScript prima di mostrare qualcosa.

CSR

1.2 Client-Side Rendering (CSR)



Server-side Rendering (SSR)

SSR è una tecnologia che genera il markup HTML della pagina lato server e la invia poi al browser dell'utente quando naviga il sito.

L'implementazione di SSR richiede più risorse al server e risulta più complessa da implementare.

Il server lavora quindi si occupa di creare un'intera pagina pronta, che verrà consegnata quando richiesta dall'utente.

Vantaggi:

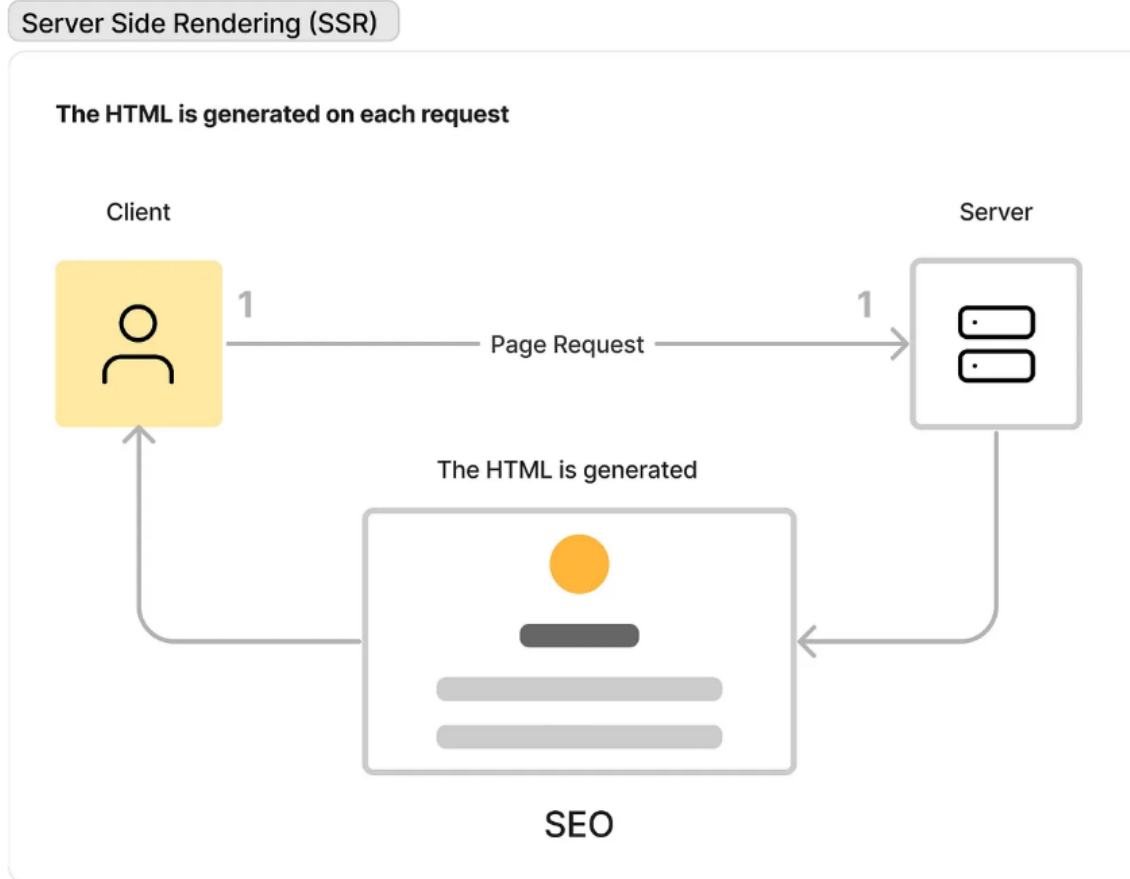
SEO migliore, cioè il ranking all'interno delle ricerche browser.

Svantaggi:

caricamento iniziale più lento.

SSR

Dal momento che il contenuto è completamente renderizzato sul server, i motori di ricerca possono scansionare la pagina e indicizzarla facilmente. Questo è fondamentale per i siti in cui il traffico di ricerca organico è importante.



<https://medium.com/@mohamed.mokhtari/choosing-the-right-rendering-strategy-for-your-business-ssr-csr-or-hybrid-27ab98cecca7#:~:text=Amazon%20uses%20Server%2DSide%20Rendering,watching%20after%20the%20initial%20load.>

Casi d'uso di CSR o SSR

SSR



CSR



Approcci Ibridi

Un approccio ibrido combina sia il Server-Side Rendering (SSR) che il Client-Side Rendering (CSR), utilizzando SSR per il caricamento iniziale della pagina per ottimizzare SEO e prestazioni, e CSR per gli elementi più interattivi e dinamici. Ciò consente un rendering iniziale veloce e SEO-friendly, pur consentendo l'esperienza ricca e interattiva che gli utenti si aspettano dalle moderne applicazioni web.

Approccio Ibrido

Il contenuto statico delle pagine è renderizzato sul server per garantire la SEO, mentre gli elementi interattivi come i commenti, i lettori video e gli aggiornamenti dinamici delle notizie sono gestiti dal lato client, offrendo un'esperienza utente fluida e garantendo al contempo l'indicizzazione della pagina e la sua reperibilità da parte dei motori di ricerca.

Cosa è la SEO

La SEO è l'insieme delle tecniche usate per ottimizzare un sito web affinché compaia tra i primi risultati dei motori di ricerca.

Perché esiste la SEO?

Un motore di ricerca deve:

- **Trovare** le pagine (crawling)
- **Capire** di cosa parlano (indicizzazione)
- **Valutare** quali pagine sono più utili e rilevanti per l'utente (ranking)

La SEO serve per aiutare i motori di ricerca in tutti e tre i passaggi.

Cosa è la SEO?

I tre fattori considerati dalla SEO sono:

SEO On-Page

- struttura HTML pulita, tag <title> e <meta description> , uso corretto dei tag <h1>...<h6> , contenuti chiari e originali, keyword nel testo in modo naturale ,immagini con attributo alt

SEO Off-Page

- link da altri siti (backlink), reputazione del dominio, citazioni e menzioni online

SEO Tecnica

- performance del sito, struttura degli URL, mobile-first indexing, sicurezza (HTTPS)

Approcci passati...

Inizialmente, tutto il lavoro avveniva sul server. Linguaggi come **PHP**, **ASP** (poi ASP.NET) e **JSP** (Java Server Pages) dominavano la scena.

ASP

- La tecnologia ASP (*Active Server Pages*) consente di usare i linguaggi di scripting dal lato del server: il codice VBScript o C# viene eseguito direttamente sul server, al momento dell'invio della pagina HTML che lo contiene.
- Il codice del programma si trova nel corpo della pagina ed è racchiuso dai delimitatori `<%` e `%>`

PHP

- Le pagine PHP possono essere richieste direttamente al web server e innescano l'elaborazione lato server, mentre in output stampano codice HTML che va al client.
- Il codice PHP è annegato all'interno di una pagina HTML attraverso un apposito tag `<?php ... ?>`
 - Il file ha estensione .php e non più .html

JSP

- La tecnologia JSP (*Java Server Pages*) è utilizzata per costruire web application costituite direttamente da pagine HTML dinamiche
- In queste pagine i tag HTML sono arricchiti da tag opportuni che contengono le istruzioni di programma scritte in Java.
- La pagina dinamica viene tradotta in una vera e propria servlet dall'application server.
- Sono in genere l'interfaccia della servlet

Esempio JSP

```
<%@ page language="java" contentType="text/html;  
charset=UTF-8" %>  
<html> <head> <title>Pagina JSP</title> </head>  
<body>  
<h1>Benvenuto!</h1>  
<p>Ora attuale: <%= new java.util.Date() %></p>  
</body>  
</html>
```

Applet e Servlet

- Java consente altre due modalità di elaborazione con interfaccia web:
 - *Applet*
 - *Servlet*
- Una applet Java è riferita all'interno della pagina ospitante attraverso il tag HTML5 `<object>`. L'effettiva applicazione viene spedita separatamente al client e ha una serie di limitazioni dovute alla sicurezza:
 - *Non può accedere al disco locale*
 - *Non può accedere ad alcuna periferica*
 - *Può comunicare solo con il server che l'ha mandata*
 - *Può essere autenticata dal server e, in tal caso, può accedere alle risorse locali del client*

Applet e Servlet

- Una servlet o *web application* è una vera e propria applicazione server, scritta in linguaggio Java che comunica tramite canale web attraverso l'uso diretto del protocollo HTTP dall'interno del programma.
- Le web application vengono gestite da appositi server chiamati *application server* che estendono le funzionalità di un web server standard
 - Contengono e gestiscono più web application insieme
 - Per ogni web application gestiscono sia la parte di interfaccia sia l'accesso agli archivi di dati usati dall'applicazione

Applet esempio

```
import java.applet.Applet;
import java.awt.Graphics;
public class MioApplet extends Applet {
    public void paint(Graphics g) {
        g.drawOval(50, 50, 100, 100); // Disegna un cerchio
    }
}
```

Codice applet java

```
<html>
<head>
    <title>Esempio di Applet</title>
</head>
<body>
    <h2> MioApplet Applet</h2>
    <applet code=" MioApplet.class" width="300"
height="300"></applet>
</body>
</html>
```

Codice HTML che integra Applet

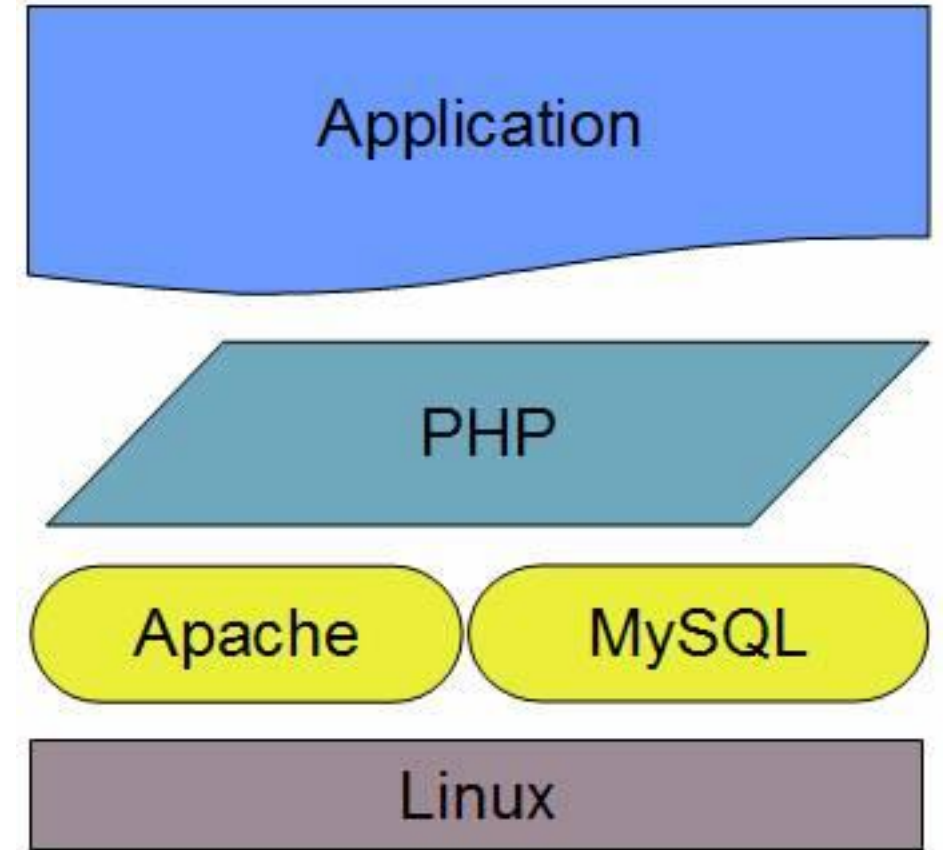
Servlet esempio

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>Ciao, questa è una Servlet!</h1>");
    }
}
```

Architettura LAMP

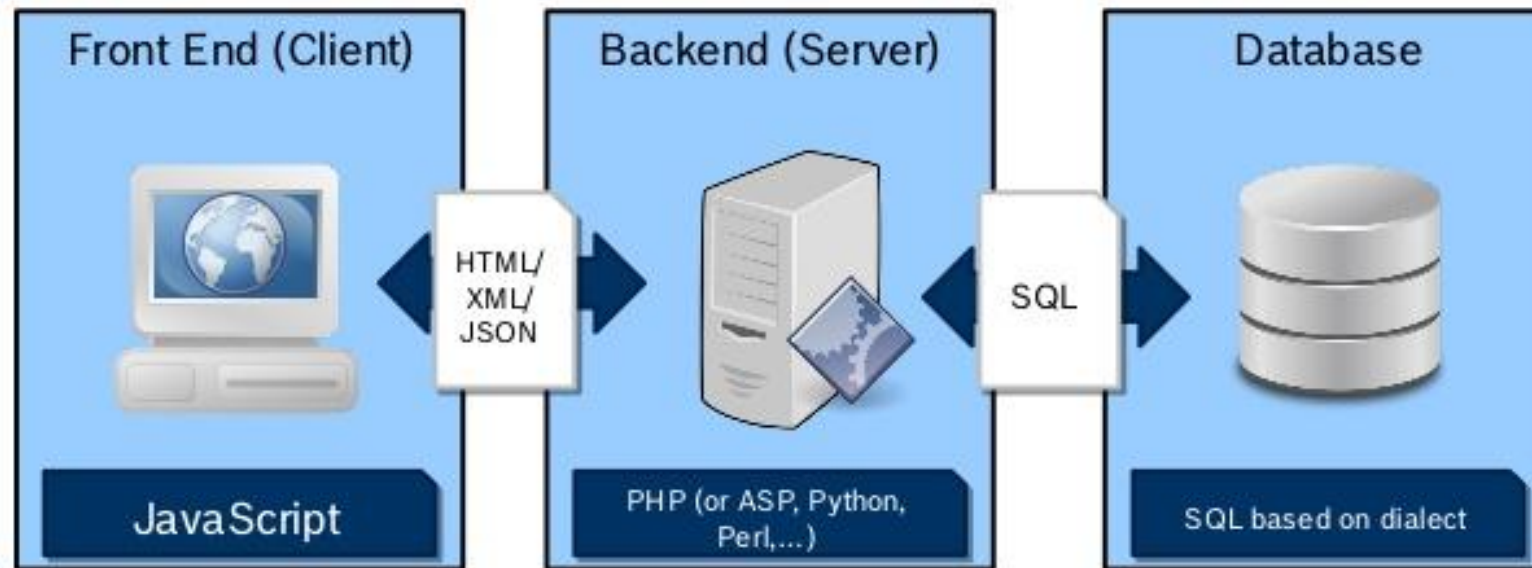
- LAMP: **L**inux, **A**pache, **M**ySQL, **P**HP (**P**erl, **P**ython)
 - WAMP per Windows
 - MAMP per MacOS
- *XAMPP* e *MAMP* sono pacchetti software diffusi per realizzare l'architettura LAMP su Windows e MacOS



LAMP funzionamento

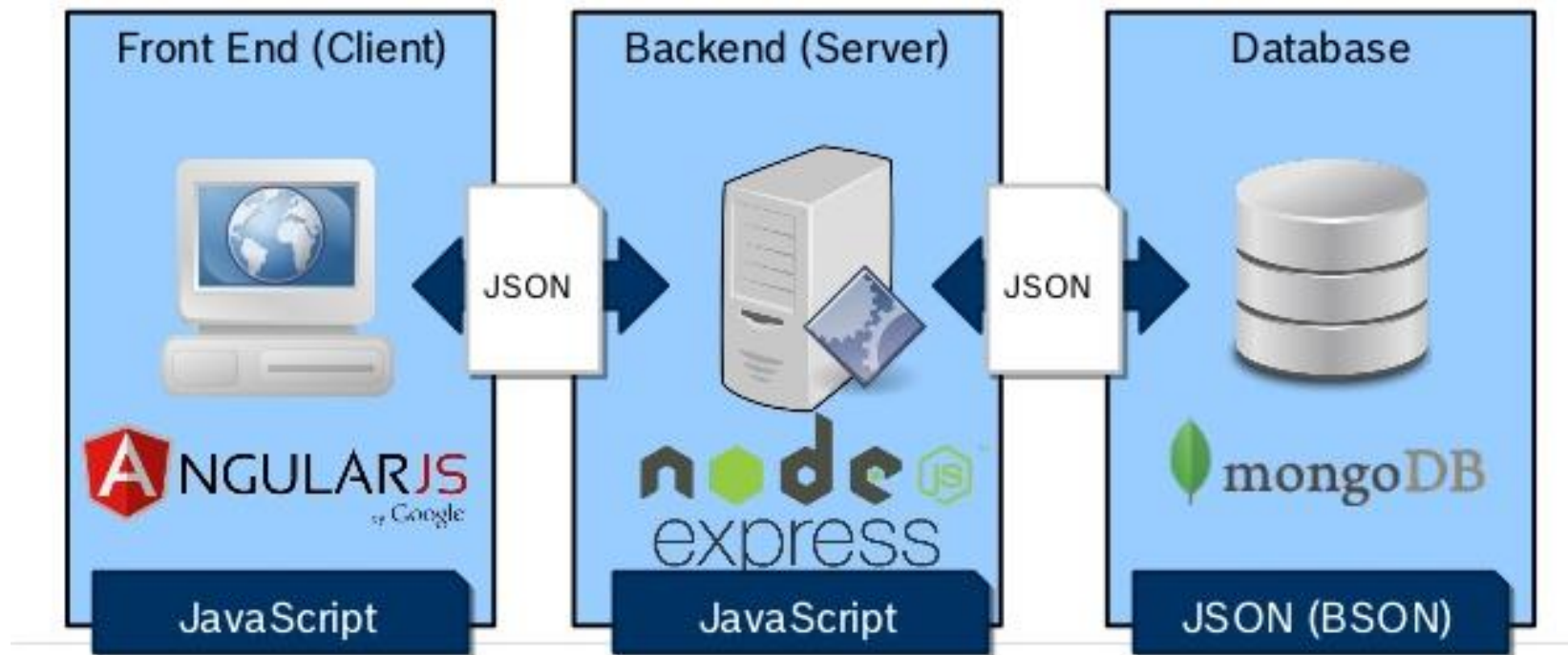
- 1.L'utente visita un sito** → Inserisce un URL nel browser.
- 2.Apache riceve la richiesta** → Il server web Apache analizza la richiesta HTTP.
- 3.PHP elabora la logica** → Se la richiesta necessita di dati, PHP (o Python/Perl) interagisce con il database MySQL.
- 4.MySQL restituisce i dati** → PHP li elabora e genera una pagina HTML dinamica.
- 5.Apache invia la risposta** → Il browser visualizza la pagina generata

LAMP: passato e presente ...



MEAN: il futuro (e anche parte del presente...)

- **MEAN**: **M**ongo DB **E**xpress.js **A**ngular.js **N**ode.js



MEAN funzionamento

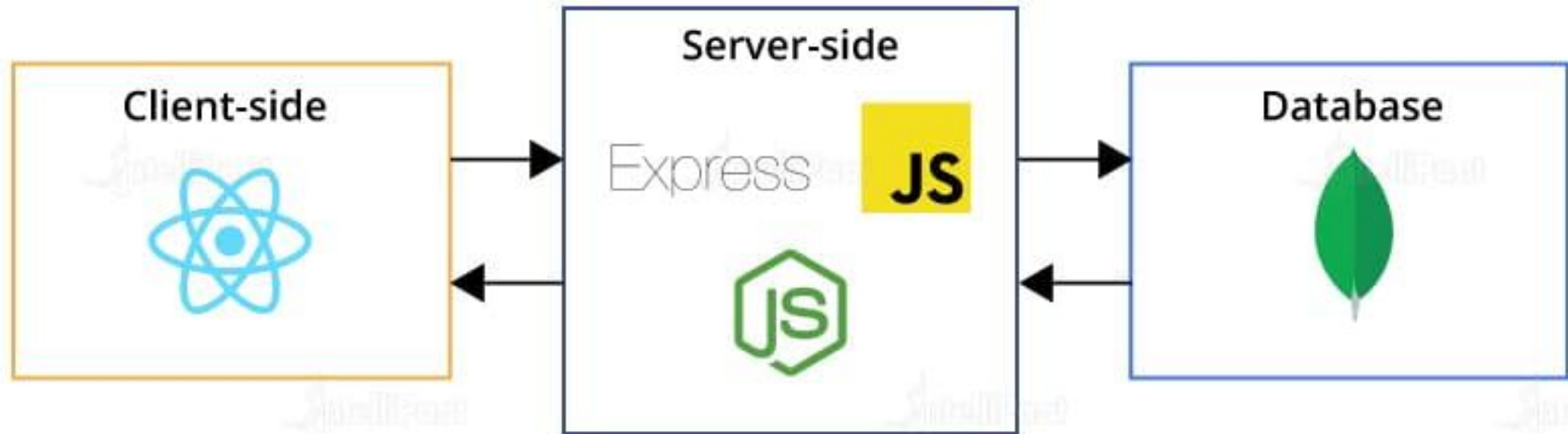
- **Come funziona?**

- 1.L'utente apre l'applicazione Angular** → Angular gestisce il frontend.
- 2.Angular invia una richiesta a Node.js** → Quando servono dati, Angular chiama il backend tramite API REST.
- 3.Express.js riceve la richiesta** → Il server Node.js con Express elabora la richiesta.
- 4.MongoDB restituisce i dati** → Express interroga MongoDB e riceve i dati.
- 5.Express invia i dati a Angular** → Angular aggiorna la UI senza ricaricare la pagina (Single Page Application).

MEAN ... o MERN?

- **MERN**: **M**ongo DB **E**xpress.js **R**ead.js **N**ode.js

The 3-tier Architecture of MERN Stack

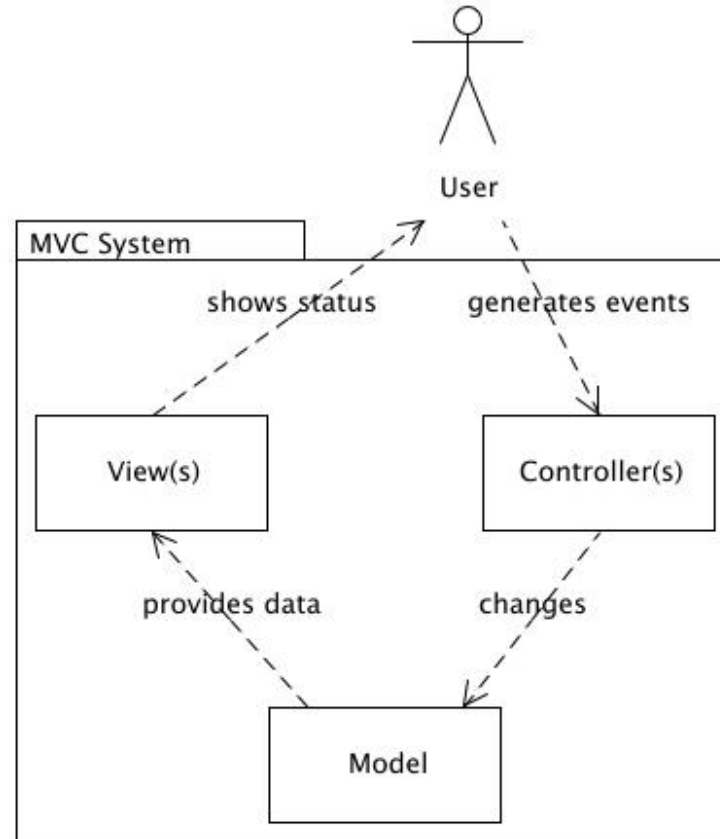


MERN funzionamento

- **Come funziona?**

- 1.L'utente interagisce con React** → React aggiorna la UI con il Virtual DOM.
- 2.React invia richieste a Node.js tramite fetch/Axios** → Quando servono dati, React chiama le API REST del backend.
- 3.Express.js riceve la richiesta** → Node.js con Express gestisce la logica server-side.
- 4.MongoDB restituisce i dati** → Express interroga MongoDB e invia la risposta a React.
- 5.React aggiorna la UI** → Senza ricaricare la pagina, la UI mostra i nuovi dati.

Considerazioni architetturali - pattern MVC

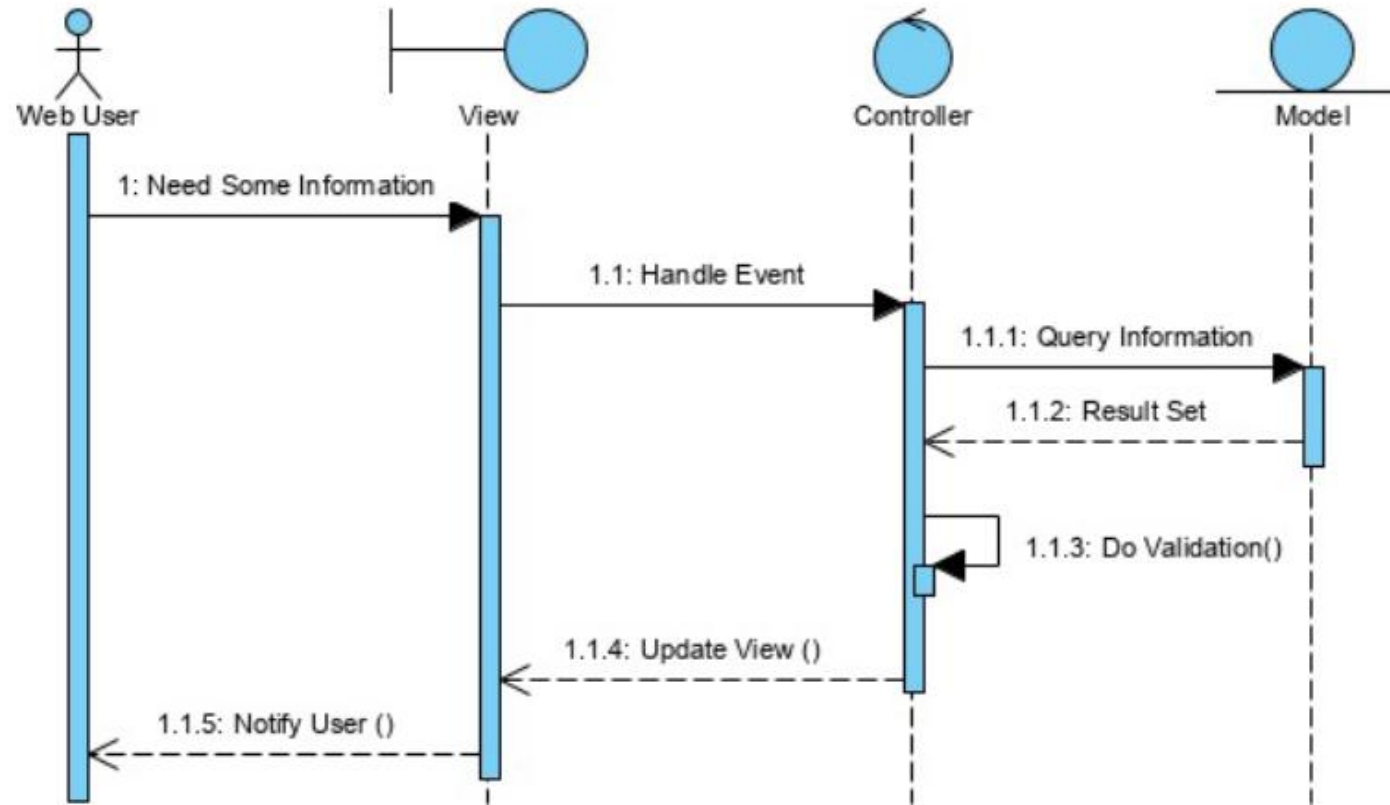


MVC funzionamento

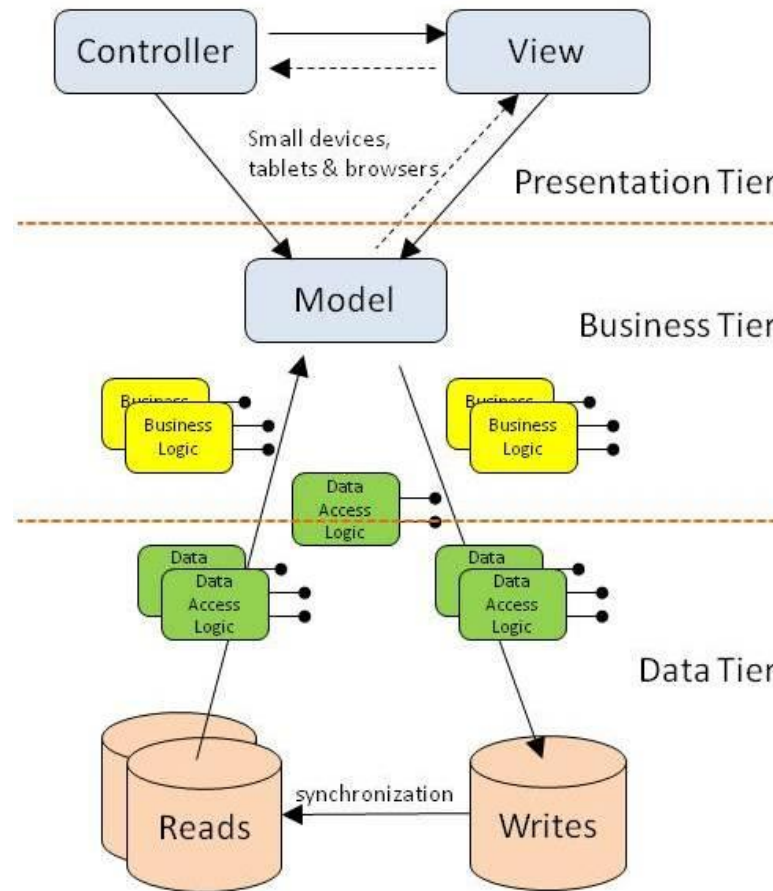
- **Come funziona?**

- 1.L'utente interagisce con l'applicazione** → Esempio: clicca su un pulsante.
- 2.Il Controller riceve la richiesta** → Interpreta l'input e decide l'azione da eseguire.
- 3.Il Model recupera i dati** → Se serve, interroga un database o esegue le elaborazioni.
- 4.Il Controller aggiorna la View** → Passa i dati alla View, che genera l'interfaccia utente aggiornata.
- 5.L'utente vede il risultato** → La View viene aggiornata nel browser o nell'applicazione.

Pattern MVC



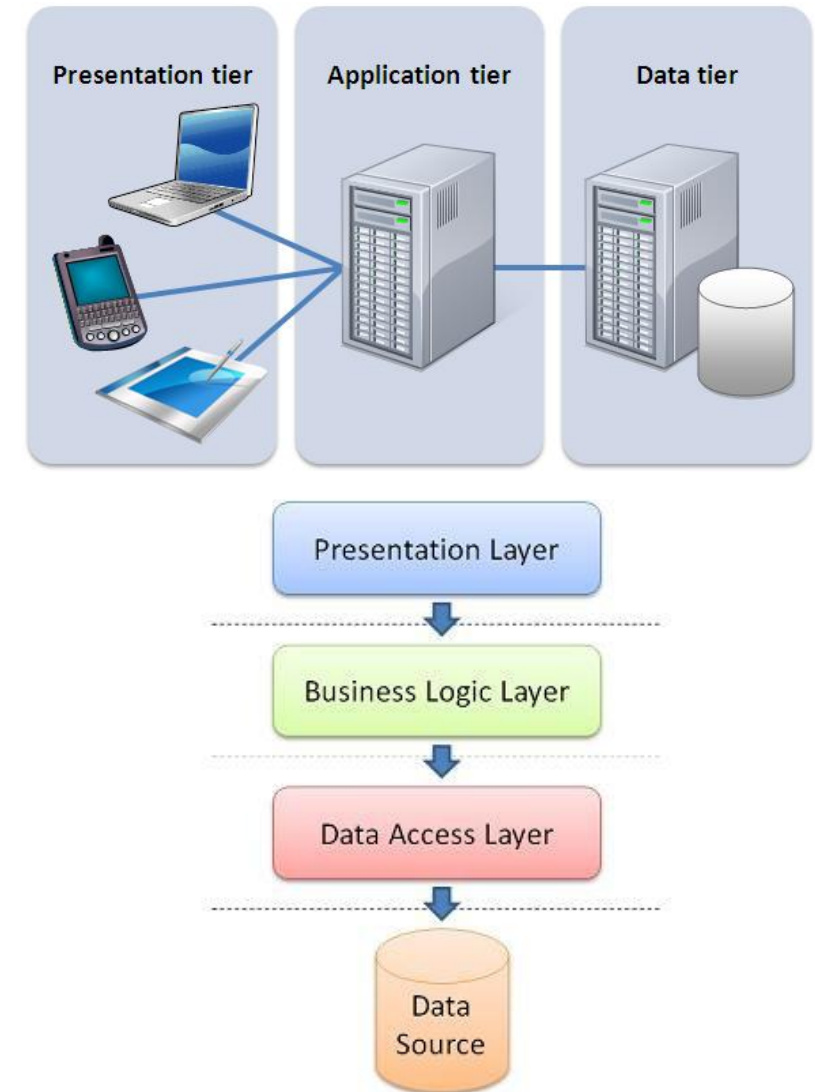
Realizzazione del pattern MVC nel modello Three Tier



Considerazioni architetturali

Modello “Three-tier”

- E' il modello cui si ispirano tutte le web application
- Il presentation tier gestisce l'interfaccia verso tutti i dispositivi utente
- L'application tier (o *Business Logic tier*) è il cuore dell'applicazione e include anche il web server e le tecnologie correlate
- Il data tier consiste nell'insieme degli archivi di dati utilizzati e le relative tecnologie di accesso (database server)



Architettura SPA + API REST

ARCHITETTURA SPA (COMPONENT BASED) + API REST

